MINOR PROJECT SYNOPSIS (July–December 2025)

Project Title:

OverFlow – A Modular System Observability and Augmentation Framework for Linux

1. Introduction

(OverFlow – A Modular System Observability and Augmentation Framework for Linux)

Modern operating systems serve as the central nervous system of digital infrastructure, managing resources, executing programs, and facilitating all user and machine interactions. While their functionality is well-documented and increasingly abstracted for user convenience, **deep observability**—the ability to transparently trace, interpret, and interact with low-level process behavior at runtime—remains limited and underutilized, especially in educational, security, and diagnostic contexts.

The traditional model of operating system introspection typically falls into two extremes:

- **Kernel-space tools** (e.g., writing kernel modules or using debugfs) that offer deep visibility but pose **high risk**, demand privileged access, and carry system stability hazards.
- **User-space tools** (e.g., top, strace, htop, or static logs) that are safe but often **limited in scope, granularity, and flexibility**.

OverFlow proposes a **new paradigm**—an **augmented user-space observability and runtime augmentation layer** that sits alongside the operating system without modifying or replacing it. Think of it as an **overlay OS**—a modular, lightweight system that monitors, extends, and interacts with the host OS processes **non-invasively**. The project name, "OverFlow," metaphorically captures this: it *flows over* the existing system, injecting awareness and capability **without rewriting its foundation**.

Core Problem: Lack of Dynamic, Safe, Fine-Grained Observability

Let's consider a common challenge: a developer wishes to monitor how a binary interacts with the system—what syscalls it makes, how memory usage evolves, or what external files it accesses. Their options are limited:

- Use strace, which is noisy, lacks filtering, and isn't live-updated
- Write a kernel module, which is dangerous and often blocked on modern systems
- Manually reverse-engineer or wrap the application, which breaks workflows

In production systems, DevSecOps teams often need to trace misbehaving services or inject runtime configuration changes without restarting processes. System administrators might want to trigger alerts when suspicious syscall patterns emerge. Educators wish to visualize how processes invoke fork, execve, or open, but can't risk exposing students to root-level system changes.

There is a clear **gap**: the need for a **modular, real-time, safe observability engine** that integrates seamlessly into the OS without breaking it.

Conceptual Solution: Legal, Transparent Augmentation Layer

OverFlow fills this gap by introducing a **meta-layer**—a framework that legally attaches to existing processes via mechanisms provided by the OS itself (e.g., ptrace, eBPF, and dynamic linking overrides via LD_PRELOAD), providing full visibility into:

- What system calls are made
- Which functions are being executed
- How files, memory, and threads are accessed
- When specific runtime conditions are triggered (e.g., memory spikes, fork bombs, syscall loops)

It doesn’t just passively observe—it allows you to **augment behavior** in real time:

- Auto-restart or suspend processes when certain patterns are detected
- Intercept and modify function outputs at runtime (e.g., redirect file reads)
- Attach plugins to inject custom logic (e.g., process shadowing or syscall filtering)

All this is done while maintaining strict **compliance with Linux user-space permissions**, ensuring that no kernel-level access is required and that operations are transparent, reversible, and safe.

Framework Vision

OverFlow is not a monolith—it's a **modular framework**, composed of:

1. **Process Monitor Core:**
   Uses ptrace and /proc to hook into active processes, trace their syscalls, and log events in structured format.
2. **Augmentation Injector:**
   Uses LD_PRELOAD to override specific glibc/system calls, allowing runtime modifications, behavioral interventions, or sandboxed redirection.
3. **Plugin Engine:**
   Developers can write lightweight plugins (in Python/C/Rust) that define triggers and responses (e.g., log this syscall, block this path, inject ENV).
4. **Visualization & API Layer:**
   A Flask-based dashboard that streams live logs and behavior summaries in an accessible UI. It also exposes APIs for automation tools or external AI modules.

Use-Cases and Ethical Boundaries

OverFlow explicitly operates within the boundaries of **academic research, security diagnostics, developer tooling, and educational visualization**. It is **not malware**, **not exploitware**, and does not engage in privilege escalation, rootkit behavior, or unauthorized process interference.

Use-cases include:

- **Blue-Team Security Research:**
  Detect syscall anomalies, monitor attack patterns in sandboxed testbeds, or teach secure programming practices by visualizing syscall behavior.
- **Developer Tooling:**
  Analyze how software interacts with system resources without source code, or dynamically tweak library calls during testing.
- **Educational Visualization:**
  Demonstrate system call flows during lectures or labs without risking system integrity. Example: showing how fork and exec spawn new processes visually.
- **Sandbox Augmentation:**
  Test legacy applications with runtime overrides or injections without recompiling them. Useful for backward compatibility and controlled stress testing.

Why OverFlow Matters

This project demonstrates the possibility of building **low-level system augmentation tools without violating ethical, legal, or security constraints.** By architecting a tool that mimics kernel-level insight while respecting user-space constraints, OverFlow becomes both a proof-of-concept and a usable system component for further research and development.

At its core, OverFlow teaches us one thing:

You don't need to hack the system to master it. You just need to observe it deeply enough.

By observing, augmenting, and respecting system behavior, OverFlow can serve as the foundation for future AI observability agents, secure sandboxed execution environments, and real-time system insight platforms—all of which are increasingly valuable in the age of distributed, containerized, and AI-augmented computing.

---

2. Project Aim

Central Aim:

To design, implement, and demonstrate a **modular, legally compliant, and non-invasive system augmentation and observability framework for Linux**—titled **OverFlow**—that enables developers, security analysts, and educators to monitor and dynamically interact with active processes in real time, entirely from user space.

Core Functionality Breakdown

OverFlow is not intended to replace or mimic an operating system kernel, but rather to operate as an **overlay agent** that attaches to an existing system. The aim is to:

1. **Observe** – passively monitor running Linux processes at a syscall level
2. **Interpret** – collect and visualize structured behavior patterns
3. **Augment** – optionally inject controlled, reversible behaviors in real time
4. **Extend** – provide a plug-and-play architecture for writing custom interventions

The project should result in a **functionally complete framework** that can be:

- Run on modern Linux systems without root
- Extended via APIs or plugins
- Understood and verified at the source code level
- Demonstrated through multiple use-cases, including security, diagnostics, and education

---

Design Principles Guiding the Aim

Principle	Description
Transparency	All monitoring and augmentation must be visible to the user; no stealth, no obfuscation.
Safety	OverFlow will operate entirely within user-space boundaries and will not use unsafe memory manipulation, privilege escalation, or kernel-level injections.
Modularity	The architecture will be layered, allowing each component (tracer, logger, injector, dashboard) to operate independently and be replaced or extended.
Reversibility	Any augmentation must be revertible without system reboot or state corruption.
Legality	Only Linux-permitted mechanisms (e.g., ptrace, LD_PRELOAD, eBPF, procfs) will be used. The tool will comply with academic and industry software security guidelines.

Operational Goals Aligned with the Aim

The system must achieve the following in execution:

1. **Syscall-Level Observability**:
   Every system call made by a process (or group of processes) must be logged in a structured, timestamped, filterable format. This includes syscall name, arguments, return values, and associated process/thread metadata.
2. **Real-Time Dashboard**:
   Visualization layer should reflect system state changes, process lifecycle events (fork, exec, exit), memory allocation patterns, and syscall frequencies.
3. **Safe Runtime Hooking**:
   OverFlow must safely inject library overrides into user-level processes using LD_PRELOAD, allowing controlled override of common functions (e.g., open, read, malloc) without breaking execution flow.

4. **Custom Augmentation Plugin System**:
   Users should be able to define custom "hooks" or plugins—for example:
   - If a process forks more than N times → trigger alert
   - If memory use exceeds X MB → inject throttling delay
   - Log every invocation of execve along with environment variables
5. **Minimal System Impact**:
   The project should measure and document its overhead on system performance (CPU/memory usage before vs. after agent attachment) to ensure deployability on real-world systems.
6. **Exportability and Reusability**:
   Logs, plugins, and system configurations should be portable. The tool should be usable as a standalone CLI utility or daemon.

Vision in One Sentence:

OverFlow aims to create a real-time, legally bound, user-space augmentation layer that empowers users to understand and interact with the operating system at a near-kernel level—without ever touching the kernel.

---

3. Objectives

Whereas the **aim** defines the vision and scope of OverFlow, the **objectives** articulate the concrete system-level deliverables that will **operationalize** that vision. These are the measurable, modular milestones that ensure the project isn't just theoretical, but *buildable*, *testable*, and *deployable*.

Each objective is designed to cover a distinct technical or functional layer of the system, aligned to software engineering best practices: *modularity, extensibility, testability, and user accessibility*.

Objective 1 – Build a Real-Time Process Monitoring Engine (Tracer Core)

Implement a core tracing engine that attaches to one or more user-level processes, intercepts their system calls in real time, and logs all relevant execution metadata.

Features:

- Use the ptrace system call to trace active processes
- Log every system call invoked: syscall name, arguments, return values, process/thread ID
- Include metadata: timestamp, duration, parent PID, file descriptors used
- Store logs in JSON/CSV format with export capability
- Handle process forking (fork, vfork, clone) and execution (execve) recursively
- Error-handling for inaccessible processes, permission errors, zombie PIDs

Evaluation Metric:

- Successful tracing of at least 90% of common Linux system calls (based on syscall table)
- Logged data integrity under rapid syscall generation (stress test)

Objective 2 – Design a Structured Logging and Query Layer

Convert raw syscall streams into a queryable format that can be filtered, sorted, and analyzed over time.

Features:

- Create structured logs with schema: { timestamp, syscall, pid, ppid, args[], retval, duration, type }
- SQLite backend for queries (or flat file JSON with live filtering via Python scripts)
- Build filtering options: by syscall type, by process, by argument patterns (e.g., filenames, ports)
- Optional export: CSV/PDF report for diagnostics or academic usage

Evaluation Metric:

- Can extract all syscalls involving file access (open, read, write) for a given process
- Can aggregate syscall counts, memory calls, file access patterns for any time window

---

Objective 3 – Develop a Runtime Augmentation Engine (Safe Function Injection)

Enable OverFlow to modify process behavior at runtime using LD_PRELOAD to override selected libc functions.

Features:

- Use LD_PRELOAD to override functions like open(), malloc(), getenv(), read(), etc.
- Allow user-defined replacement behavior (e.g., redirect open("/etc/passwd") to /dev/null)
- Must preserve original functionality where not explicitly modified
- Support revert-on-exit functionality for full cleanup

Constraints:

- No breaking of dynamically linked applications
- No modification to binaries or on-disk files
- Only applies to user-space processes with dynamic linking (no static binaries)

Evaluation Metric:

- Inject and override at least 5 system-level libc functions without crashing the host process
- Time to inject ≤ 500 ms for processes under 50MB footprint

Objective 4 – Build a Real-Time Web Dashboard (UI Layer)

Develop a Flask-based dashboard to display live metrics and logs in an intuitive, structured interface.

Features:

- Frontend (HTML + JS + D3.js): syscall frequency graphs, process tree visualizer, memory maps
- Backend: live JSON feed from tracing and augmentation engine
- System filters: by PID, by syscall, by behavior type
- Admin interface: enable/disable modules, inject plugin, download reports

Evaluation Metric:

- Dashboard latency ≤ 2 seconds under sustained tracing of 5 simultaneous processes
- Real-time chart updates for live syscalls and memory tracking

---

Objective 5 – Implement Plugin-Based Augmentation Triggers

Allow the user to write or load simple Python/C plugins that define custom conditions and actions.

Examples:

- If memory usage > 500MB → log warning or kill process
- If execve() is called with suspicious path → block or redirect
- Inject delay into any call to read() if called more than 1000 times/second

Plugin API:

- Trigger condition function (evaluated on each syscall)
- Action function (e.g., log, override, kill, inject env, send socket message)
- Security sandbox: plugins run in isolated subprocess, have resource limits

Evaluation Metric:

- At least 3 custom plugins can be created and attached without modifying OverFlow core
- Plugin sandbox survives plugin crash without compromising parent process

---

Objective 6 – Integrate eBPF-Based System Metrics

Extend observability by tapping into kernel-level performance metrics using eBPF (without custom kernel modules).

Features:

- Monitor CPU usage by process
- Track syscall latency and I/O bandwidth over time
- Collect context switches, page faults, and memory allocations (if permitted)

Evaluation Metric:

- eBPF probes return real-time data in <1s delay
- Metrics are visualized alongside syscall logs on dashboard

Objective 7 – Benchmark, Test, and Optimize Performance

Ensure OverFlow does not introduce unacceptable overhead to the system and can be safely run in real-world conditions.

Tests:

- Run OverFlow on typical system workloads: code compilation, video playback, network tools
- Measure CPU, RAM, syscall delay overhead with and without OverFlow
- Stress test with process forking loops and memory spikes

Optimization Metrics:

- ≤ 10% CPU overhead under normal tracing load
- ≤ 100MB RAM usage by the OverFlow agent
- Zero crashes or memory leaks after 4+ hours of continuous operation

---

Completion Deliverables

By the end of the project, the following must be completed:

- Full codebase under version control (GitHub or GitLab)
- Executable build for Linux x86_64
- Sample plugins + documented API
- Demo video showing tracing, augmentation, and dashboard features
- Final report documenting system design, implementation details, use-cases, test results

4. Tools and Technologies

The OverFlow system spans multiple abstraction layers—from **low-level process tracing** and **dynamic function interception**, to **data storage**, **visual analytics**, and **plugin sandboxing**. To maintain security, portability, and modularity, each component is implemented with tools purpose-fit to its technical role.

This section outlines the **technology stack**, along with the **reasoning** for each selection and how it integrates into the overall architecture.

Core Programming Languages

Languag	Purpose	Justification
C	System call interception, LD_PRELOAD override modules, low-level performance tracing	Direct access to POSIX APIs and syscall interface. Near-zero overhead.
Python	Dashboard backend, plugin system, logging engine, orchestration scripts	Rapid prototyping, rich libraries (Flask, sqlite3, psutil, subprocess). Dynamically extensible.
Rust *(optional)*	Safety-critical plugins, high-performance logging modules, optional API layer	Ensures memory safety and zero-cost abstractions. Adds robustness to plugin execution. Optional if time permits.
Bash	Deployment scripts, process launching, system setup automation	Lightweight shell automation for setting up test environments and rapid control testing.

System Interfaces & OS Mechanisms

Interface	Role	Details
ptrace	Process attachment and syscall interception	Core mechanism for tracing running processes. Captures syscall entry/exit, arguments, and return values. Controlled via C interface.
LD_PRELOAD	Dynamic function override	Injects custom wrappers for library calls like open, malloc, read, etc. Used for runtime augmentation. Requires no binary modification.
/proc filesystem	Process metadata retrieval	Used to extract runtime information like memory usage, file descriptors, executable path, command-line arguments. No root required.
eBPF (optional module)	System-wide performance metrics	Access kernel-level metrics (context switches, syscall frequency, I/O bandwidth) in a safe, programmable way. Tools like bcc or libbpf can be used with root privileges or in limited scope.
inotify	Filesystem-level event tracking	Optional module for monitoring specific files accessed by traced processes (e.g., track when sensitive config files are opened).

Visualization and Dashboard Layer

Component	Role	Justification
Flask (Python)	REST API + Web dashboard backend	Lightweight Python web framework ideal for quick server-side rendering and JSON API streaming.
SQLite	Local data storage	Easy integration with Python. Lightweight, fast, and doesn't require a separate DB server. Used to store logs and behavior traces.
Chart.js or D3.js	Frontend data visualization	Used to graph system call frequencies, process trees, memory timelines, and other time-series metrics in a dashboard.
HTML/CSS/JS (Bootstrap)	Frontend UI layer	Rapid styling and responsive layout with prebuilt templates. Ensures clean visuals for live demonstrations.

Plugin Engine

Feature	Technology	Justification
Custom Hooks	Python or C	Each plugin defines conditions and responses to system events. Exposed as callable functions in isolated sandboxes.
Sandbox Execution	multiprocessing, resource, subprocess	Ensures plugins cannot crash the parent OverFlow process. Uses soft CPU/ memory limits.
Communication	IPC via pipes or shared memory	Fast exchange of syscall metadata between tracer and plugin hooks. Controlled serialization and validation.

Development and Testing Tools

Tool	Use
GDB	Manual debugging of syscall wrapper logic
Valgrind	Memory leak detection in C modules
Wireshark (optional)	Monitor if injected processes attempt network operations (useful in certain plugins)
Docker	Create isolated containers for testing potentially dangerous plugins or tracing multiple environments
Git	Version control and release management
Make/CMake	Build automation for native modules

Linux Environment Requirements

Requirement	Specification
Distribution	Debian-based or Arch Linux (tested on Ubuntu 22.04, Arch 2025.07)
Kernel Version	≥ 5.4 (for eBPF compatibility)
Architecture	x86_64 (optional ARM64 compatibility if time permits)
Privileges	No root required for core tracer, dashboard, and LD_PRELOAD modules. Root required only for eBPF integration.
Dependencie	gcc, python3, pip, sqlite3, libc-dev, gdb, make

Stack Design Philosophy

Principle	Stack Support
Low-Level Control	Achieved via C, ptrace, and LD_PRELOAD for direct syscall and memory interactions
Safety & Sandboxing	Python sandboxed plugins, optional Rust modules
Visibility & Demonstration	Dashboard enables real-time demonstration of system activity
Portability	Works on commodity Linux systems, no kernel patching or root installation required for core features
Extensibility	Plugin API, REST endpoints, and data export modules designed for future AI observability integrations

---

5. Expected Outcomes

The success of OverFlow is determined not by conceptual ambition alone, but by the **completion and verification of tangible, testable results** across multiple subsystems. The following outcomes represent the **minimum viable system** you must produce to demonstrate real-world viability and academic value.

Each outcome will be accompanied by a **proof mechanism**—something observable by your professors, usable by developers, or exportable for presentation.

---

1. Fully Functional Process Tracing Engine

Description:

A system-level component capable of attaching to one or more live Linux processes (owned by the user) and intercepting their system calls in real-time, using ptrace.

Deliverables:

- Real-time syscall log feed, structured in JSON
- Timestamps, syscall names, arguments, return values
- Coverage of common syscall families: file I/O, memory, process control, IPC, networking
- Resilience against process forking, termination, or exec-replacement

Proof of Completion:

- Run OverFlow on a live terminal session or browser process
- Show real-time logs in console and dashboard
- Export 1+ minute trace into file and analyze syscall frequency graph

---

2. Live Dashboard with Visualization Layer

Description:

A browser-accessible UI served by a local Flask server, streaming real-time data from the tracer and displaying visual insights into process behavior.

Deliverables:

- Live-updating syscall frequency chart

- PID tree with expandable child processes
- Filter and search by syscall type or PID
- Display of memory usage, file accesses, and open descriptors (optional)

Proof of Completion:

- Access dashboard on localhost:5000
- Inject sample process (e.g., Python script) and observe changes live
- Switch between trace targets and analyze deltas

---

3. Runtime Augmentation Module (LD_PRELOAD Subsystem)

Description:

An LD_PRELOAD-based library that can override selected libc functions in dynamically linked applications at runtime, allowing controlled behavior injection.

Deliverables:

- Wrap and override open(), malloc(), read(), getenv() functions
- Redirect or modify return values in real time
- Revert changes on process exit
- Maintain stability even when injected into heavy applications (e.g., browser)

Proof of Completion:

- Compile and run a test binary with OverFlow's LD_PRELOAD layer
- Show output difference with and without OverFlow (e.g., redirected file path)
- Demonstrate override of malloc() with a logging wrapper

---

4. Plugin Execution Engine (Custom Event Hooks)

Description:

An API and runtime allowing users to write Python or C plugins that define logic to execute on specific conditions (e.g., memory usage threshold, syscall pattern, command-line match).

Deliverables:

- Plugin interface for defining on_syscall() and on_event() hooks
- Sample plugins (e.g., auto-log, kill, inject env, trigger alert)
- Isolation via multiprocessing or restricted subprocess

Proof of Completion:

- Create a plugin that kills a process if it forks > 10 times
- Create a plugin that logs every use of execve()
- Demonstrate safe crash recovery from faulty plugin

---

5. Augmentation Trigger System

Description:

A runtime component that activates augmentations or logs when defined conditions are met—thresholds, behavior anomalies, or time-based triggers.

Deliverables:

- Trigger templates (e.g., memory > X, syscall Y > Z/sec)
- Plugin-trigger linkage via config
- Alert message/log on trigger event

Proof of Completion:

- Plugin to alert when rm command is executed
- Plugin to introduce delay if file I/O exceeds threshold
- Trigger auto-email/log when process reaches 500MB

---

6. Extensibility Mechanism

Description:

A simple but powerful interface to extend OverFlow via:

- API endpoint calls (for automation tools)
- CLI flags and modules (for scripting or chaining)
- Plugin registration and config system

Deliverables:

- CLI syntax for injection: overflow --trace PID and --plugin x.py
- Plugin loader that validates, executes, and unloads
- Future-ready API spec (documented JSON format)

Proof of Completion:

- Write a 20-line Python plugin and load it dynamically

- Use CLI to trace process, attach plugin, and export log—all without dashboard

---

7. Documentation and Final Report

Description:

A complete, readable, logically structured documentation of:

- System design, architecture diagrams
- Installation and usage instructions
- Plugin API reference
- Benchmarking results

Deliverables:

- Markdown/LaTeX/PDF report, 15–25 pages
- README on GitHub with instructions
- API + plugin docs as standalone HTML page

Proof of Completion:

- Submit printed and digital report
- Professors can run system using docs alone
- Classmates can test with sample plugins

---

8. Performance Benchmark Report

Description:

An empirical report on the system's runtime impact on monitored processes and system load.

Deliverables:

- RAM and CPU usage before and after tracing
- Latency impact on system calls (microseconds overhead)
- Dashboard rendering delay under load
- Stability test under 1–2 hour sustained operation

Proof of Completion:

- Benchmark results (graph/table form)
- Stress test logs from memory/cpu-intensive process tracing

6. Scope of Application

The value of a system is not limited to what it does, but in **where**, **for whom**, and **how** it can be used. This section outlines the **operational and contextual scope** of OverFlow—clarifying its use-cases across different user personas, environments, and industries. It also anticipates its future evolution and outlines the ethical and legal boundaries within which it operates.

A. Primary Domains of Application

---

1. Developer Tooling

OverFlow functions as a runtime introspection tool for developers, particularly in debugging black-box binaries, legacy software, or reverse-engineering behavior.

Key Features for Developers:

- Monitor system calls of a compiled binary without source code access
- Override function behavior during runtime (e.g., redirect file paths, log memory usage)
- Test how software reacts to runtime alterations without rebuilding or restarting
- Visualize how a process spawns threads or sub-processes (fork, exec, clone)

Example Scenarios:

- A developer tests how a legacy C application reads config files and wants to intercept its open() calls.
- An engineer debugging a multithreaded program watches for deadlocks via syscall sequences.
- A DevOps engineer benchmarks syscall density of a web server under load.

---

2. Security Research & Blue Team Operations

OverFlow can function as a legally compliant, user-space observability layer for security audits, anomaly detection, and basic behavioral sandboxing.

Key Features for Blue Team:

- Log syscall patterns and trigger alerts on known malicious behaviors
- Observe process spawning patterns that might indicate privilege escalation attempts
- Plug in heuristic rules that block certain calls (e.g., delete operations on protected directories)
- Provide runtime visibility into unknown or untrusted binaries in a controlled environment

Example Scenarios:

- A blue team member monitors an executable for suspicious behavior without rootkit-level risk.
- A security researcher wants to detect if a process is silently executing shell commands via execve.
- A SOC team traces file deletions across processes in real time for early incident detection.

---

3. Educational Use (OS and Cybersecurity Labs)

OverFlow provides a rare opportunity to **see the OS "think"**—an invaluable resource for students learning about system architecture, memory, and runtime control flow.

Key Features for Education:

- Demonstrate process lifecycle (fork, exec, exit) visually
- Show real-time syscall flows from a simple C program or shell command
- Inject controlled failures or modifications to teach robust design
- Observe memory allocation patterns, file descriptor behavior, and basic sandboxing

Example Scenarios:

- An operating systems class watches how user-space processes invoke read, write, mmap.
- A reverse engineering lab steps through syscall logs to understand binary logic.
- A teacher uses OverFlow to simulate faulty memory behavior and shows students how to catch and handle it.

---

4. Diagnostic & Observability Layer for Production Systems *(future-ready)*

While OverFlow is a user-space academic tool, its architecture can be extended into DevOps and site reliability tooling.

Possibilities:

- Pair OverFlow with Prometheus/Grafana to generate live syscall observability charts
- Feed logs into alert systems (Slack, email, custom dashboards)
- Generate periodic process behavior reports for compliance or audit logs
- Use plugin-based auto-responses for failure patterns (e.g., restart on crash, notify on fork bomb)

Example Scenarios:

- A developer tests how a CI/CD pipeline spawns subprocesses and logs them via OverFlow.
- A systems engineer watches syscall latency spikes under high disk I/O loads.
- OverFlow is paired with ML plugins to auto-detect behavior drift in services.

B. Ethical & Legal Scope

OverFlow is deliberately designed to remain within **strict ethical and legal boundaries**. It is not a penetration tool, nor a surveillance rootkit. It does not elevate privileges or obfuscate behavior.

Principle	Enforcement
Legality	Uses only Linux-sanctioned user-space APIs like ptrace, LD_PRELOAD, eBPF, /proc
Transparency	All operations are logged, all augmentations declared, and user permission required
Non-malicious	No spyware behavior, no privilege escalation, no kernel tampering, no keylogging or data theft
Research/Education Only	Designed for blue-team and academic usage, not for exploitation or black-box intrusion
Reversibility	No permanent system modifications; all injected behavior is cleaned up on process exit

---

C. Future Expansion Pathways

OverFlow is built modularly, allowing for smooth vertical expansion into more complex roles in the future.

1. AI-Augmented Observability

- Use LLMs or anomaly-detection ML models to analyze syscall patterns in real time
- AI agent can suggest behavioral profiles or detect ransomware-like activity

2. Container-Aware Tracing

- Extend compatibility with Docker and Podman containers
- Trace processes within namespaces with limited privileges

3. Distributed OverFlow

- Push syscall traces from remote agents to a central OverFlow collector
- Compare and correlate system behavior across a network

4. Cloud Integrations

- Export alerts or traces to logging tools (ELK Stack, Datadog, CloudWatch)
- Integrate with CI/CD systems to test behavior of new binaries in runtime sandbox

---

D. Who Will Use OverFlow?

Persona	Use-Case
Undergraduate Student	Learning syscalls, writing plugins, tracing simple C apps
Researcher	Building sandbox environments to analyze unknown binaries
Security Engineer	Monitoring real-time syscall behaviors during blue-team simulations
Developer	Debugging how a compiled binary accesses files, memory, or subprocesses
Educator	Demonstrating OS internals to students with no root risk

---

7. Gantt Chart (Execution Timeline)

The project will be executed over a period of **13 academic weeks**, broken into clearly defined **phases**: Setup, Core System Build, Integration, Testing, Documentation, and Evaluation.

Each phase contains focused tasks mapped to specific modules (Tracer, Injector, Dashboard, Plugins, etc.), ensuring progressive build-up and clean modular isolation.

A. Weekly Execution Table

Week	Task / Milestone	Output / Deliverable
Week 1	Finalize project scope and supervisor guidance Conduct deep technical literature review Identify toolchains and OS constraints	Initial architecture draft, finalized project problem statement
Week 2	Build base process tracer using ptrace Test with small binaries (e.g., ls, cat, compiled C scripts)	Syscall tracer prototype (manual log output)
Week 3	Expand tracer to extract syscall metadata Build structured JSON/CSV logger Add support for child processes (fork, exec)	Tracer v1.0 + logger engine with CLI
Week 4	Implement LD_PRELOAD-based override for at least 3 functions Test injection stability across programs Add command-line loader interface	Injection module (malloc/open/read override)

Week	Task / Milestone	Output / Deliverable
Week 5	Begin dashboard: Flask server setup Basic UI with process list, syscall count chart Connect logger output to web UI	Flask + dashboard v0.5 live stream
Week 6	Integrate dashboard with live metrics: syscall frequency, PID info, command-line args Add search + filter support	Real-time observability interface (minimal latency)
Week 7	Build plugin loader API Create 3 default plugins: (1) Kill-on-condition, (2) Alert-on-syscall, (3) Delay-on-memory	Plugin engine base, sample plugins loaded via config
Week 8	eBPF integration (optional) Visualize context switches, memory, I/O (if permitted) Add plugin-level dashboard triggers	Optional eBPF dashboard extension
Week 9	Begin stress-testing: long sessions, memory-intensive processes, multiple traces Add crash recovery and fail-safes	Trace stability verified; plugin sandboxing hardened
Week 10	Add export features: PDF/CSV report generation Finalize plugin API docs Polish UI layout (mobile + desktop)	OverFlow CLI + dashboard v1.0 release
Week 11	Begin report documentation: Architecture diagrams, annotated code, plugin guide Run benchmarks on syscall latency + overhead	Drafted report + benchmark data graphs

Week	Task / Milestone	Output / Deliverable
Week 12	Finalize system Record demo video + screenshots Practice internal presentation + resolve bugs	Complete working demo, internal evaluation
Week 13	External submission Final report printing Final viva / evaluation by jury	Submission package + presentation file

---

B. Milestone Breakdown (By Module)

Module	Milestone	Expected Completion
Tracer Core	Captures syscalls, logs structured output, forksafe	Week 3
Injection Layer	3+ function overrides (malloc, open, getenv), stable LD_PRELOAD loader	Week 4
Dashboard UI	Real-time streaming with graphs, filters, PID view	Week 6
Plugin System	Load 3+ plugins, execute safely on events, CLI attach/detach	Week 7
eBPF Addon	Optional context-switch + syscall latency probe (with root)	Week 8
Export Engine	Logs → CSV, PDF summaries, data charts	Week 10
Docs + Demo	Final writeup, annotated screenshots, narrated walkthrough	Week 12

C. Project Management Notes

- **Daily Logs:** Maintain a Notion/GitHub Projects board to track task completion, test results, bug logs
- **Weekly Review:** With project supervisor every Saturday (status + new risks)
- **Version Tags:** Git tagging for each subsystem milestone (v0.1, v0.5, v1.0)

D. Evaluation Readiness

By Week 12, OverFlow will be:

- Fully demoable via dashboard or CLI
- Run on a clean Ubuntu system without root
- Loaded with at least 3 plugins, with metrics and logs exportable
- Able to attach to real processes like gedit, firefox, htop, and trace live syscall graphs

---

8. Team Structure and Roles

OverFlow is a modular, multi-layered system requiring collaboration across different technical areas—low-level C programming, Python-based web systems, UI/UX, and plugin sandboxing. To ensure clarity, each team member will be assigned to a **primary responsibility domain** while maintaining overlap for code review and integration.

Team Size: 3 Members

Final team member names and roll numbers to be submitted officially as per university guidelines.

Proposed Role Distribution

Role	Responsibility Scope	Skillset Required
System Architect & Tracer Engineer	- ptrace core - Syscall mapping - JSON/CSV logger - Fork/exec tracing - Performance testing	C, Linux internals, process management, Rust (optional)
Runtime Injector & Plugin Lead	- LD_PRELOAD override - Plugin loader design - Event trigger engine - Plugin API documentation	C, Python (multiprocessing), debugging.
Dashboard & Visualization Engineer	- Flask server - Dashboard UI - Graphs (Chart.js/ D3.js) - Export/report module - UX tuning	Python (Flask), JS, HTML/CSS, SQLite.

Collective Responsibilities

All members will:

- Maintain version control via Git
- Participate in integration testing and crash handling
- Contribute to the final report, video, and presentation
- Rotate during weekly internal reviews to cross-train

Guidance and Mentorship

The team will be mentored by a faculty guide from the Department of Computer Science and Engineering, UIET, Panjab University. Weekly check-ins will include technical updates, code reviews, and evaluation of risks or architectural changes.

---

9. Compliance and Ethics

OverFlow is designed with **strict adherence to ethical, legal, and institutional boundaries**. While the system operates at a deep level of observability and runtime augmentation, it **does not and will not** cross into unauthorized access, privilege escalation, surveillance, or malicious use.

This section explicitly outlines the ethical safeguards, operating constraints, and compliance mechanisms that ensure OverFlow is suitable for academic, developmental, and research use.

A. Legal Operation Boundaries

OverFlow leverages only **legally permitted interfaces** provided by the Linux operating system:

Mechanism	Legal Justification
ptrace	Official syscall used by debuggers (e.g., gdb); tracing permitted for user-owned processes
LD_PRELOAD	Dynamic linker feature supported by glibc; used for injection of user-specified override libraries
/proc/ filesystem	User-accessible process metadata, used by almost all Linux tools (top, htop, ps)
eBPF (optional)	Linux-sanctioned tracing framework for kernel-level visibility, used with root permissions
inotify	Used for legal file-watching in user directories

B. What OverFlow Will Not Do

Action	Reason for Prohibition
Elevate privileges	No setuid, sudo, or kernel module interaction allowed
Hook kernel modules	Entire system operates in user space
Trace other users' processes	Limited by ptrace_scope in /proc/sys/kernel/yama
Install persistent backdoors	No background agents, no autorun scripts, no boot hooks
Modify binaries or system files	No static patching, no binary rewriting
Access raw network traffic	OverFlow is syscall-level only; it does not sniff packets or bypass NIC constraints

---

C. Security & Reversibility Safeguards

- **Reversible Augmentations:** All LD_PRELOAD changes revert automatically on process termination. No file or kernel state is altered.
- **Plugin Isolation:** Plugins run in subprocesses with resource limits. Crashes in a plugin do not affect host process or dashboard.
- **No Data Exfiltration:** Logs stay local. No network communication is used unless explicitly configured by the user (e.g., sending alerts to a webhook).
- **No Hidden Behavior:** All injected functions, triggers, and plugins are declared, logged, and can be audited.

D. Academic and Institutional Alignment

OverFlow is designed for and limited to the following use-cases:

- Academic operating system education
- Legal security research (blue team)
- Developmental diagnostics of binaries or legacy code
- Demonstrable learning of Linux internals and process behavior

This system **does not qualify** as penetration testing, red-team offensive tooling, or malware simulation.

E. Attribution and Licensing (For Open Release)

Should OverFlow be published post-submission:

- It will be released under the **MIT License** or **GNU GPL v3**, depending on university policy
- All injected behaviors and plugins will be documented
- A clear disclaimer will state that the tool is for **educational and research purposes only**

With this, OverFlow maintains full **transparency, legality, and academic integrity**, aligning with UIET's project guidelines, national cybersecurity norms, and global best practices in ethical computing.

---

10. Submission Details and Packaging

This section outlines the **format**, **medium**, and **contents** for final OverFlow project submission to ensure smooth evaluation, reproducibility, and alignment with departmental expectations.

A. Deliverables Package Structure

All project artifacts will be submitted in both **soft (digital)** and **hard (printed)** formats. The structure of the submission folder will be standardized for reviewer clarity:

```
OverFlow_Project/
│
├── Report/
│   ├── OverFlow_Final_Report.pdf
│   ├── Architecture_Diagrams/
│   └── API_Documentation.pdf
│
├── Demo/
│   ├── Overflow_Demo_Video.mp4
│   ├── Screenshots/
│   └── Test_Cases_Results.pdf
│
├── Code/
│   ├── tracer/            ← ptrace engine
│   ├── injector/          ← LD_PRELOAD modules
│   ├── dashboard/         ← Flask UI + frontend
│   ├── plugins/           ← Sample plugins
│   ├── export_tools/      ← PDF/CSV logger
│   ├── Makefile           ← Build automation
│   └── README.md          ← Setup + usage
│
├── Gantt_Chart.xlsx
├── License.txt (MIT or GPL)
└── ProjectSynopsis.pdf
```

B. Hard Copy Submission (Printed File)

The following printed documents must be submitted:

1. Approved **Project Synopsis** with guide signature
2. Printed copy of the **Final Report** (spiral-bound, 15–25 pages)
3. **Gantt Chart (color print)**
4. Project team cover page with:
    - Project title
    - Roll numbers, names
    - Guide name and department
    - Submission date
5. Declaration of original work

C. Google Classroom / Digital Submission

All digital materials will be submitted on Google Classroom or the project submission portal as a single .zip or .rar archive (max size ≤ 500 MB), containing:

- Final report PDF
- Gantt chart Excel
- Codebase (with comments and structure)
- Demo video (≤ 3 min, showing features and architecture)
- Screenshots of dashboard and plugin runs
- README with installation + usage

D. Timeline and Contact

- **Final Submission Deadline:** 04.08.2025 (soft + hard)
- **Internal Evaluation:** Week 12
- **External Evaluation & Viva:** Week 13 (presentation + demo)

Faculty Guide: [Name to be confirmed]
Project Lead Contact: [Student email & phone]
